

Resolução de Exercícios: Teoria Flutter

Disciplina: Desenvolvimento Mobile

Foco: Questões Teóricas (Práticas Omitidas)

1. Descreva o que é flutter, Dart, Widgets e a diferença para o Android.

Flutter é um framework de UI de código aberto do Google para criar apps multiplataforma. **Dart** é a linguagem de programação usada por ele. **Widgets** são os blocos construtivos da interface (tudo é um widget). A **diferença para o Android** é que o Android nativo usa Java/Kotlin e componentes do próprio sistema, enquanto o Flutter usa Dart e desenha sua própria interface na tela, podendo ser exportado tanto para Android quanto para iOS.

2. Tudo no Flutter é criado com widgets. Cite quais são as principais e dê exemplos:

Os widgets são divididos principalmente em **Layout** e **Visuais/Interativos**. Exemplos: **Container** (caixa estilizável), **Row** e **Column** (alinhamento horizontal/vertical), **Text** (textos), **Scaffold** (estrutura de tela) e **ElevatedButton** (botões).

3. Todos os widgets devem conter um método denominado build. Qual o objetivo e dê exemplo de como usar?

O objetivo do método **build** é construir e retornar a interface gráfica do widget. Ele é chamado sempre que a tela precisa ser desenhada. Exemplo:

```
@override Widget build(BuildContext context) { return Text('Olá, Mundo!'); }
```

4. No contexto de desenvolvimento de soluções mobile com flutter o que são container?

O **Container** é um widget versátil usado para estilizar, posicionar e dimensionar outros widgets. Com ele é possível adicionar cores de fundo, margens (margin), preenchimentos (padding) e bordas aos elementos filhos.

5. Qual a diferença entre aplicações nativas e híbridas? Dê exemplos.

Nativas: Feitas especificamente para um sistema usando suas ferramentas oficiais (Ex: Swift para iOS). Possuem performance máxima. **Híbridas/Multiplataforma:** Feitas com um código único que roda em vários sistemas (Ex: Flutter, React Native). Economizam tempo de desenvolvimento, pois um único código atende Android e iOS.

6. Quais são os principais padrões de arquitetura de software para apps móveis?

Os mais conhecidos são **MVC**, **MVP** e **MVVM**. No desenvolvimento específico com Flutter, ganham muito destaque o padrão **BLoC** (Business Logic Component) e a **Clean Architecture** para gerenciar o estado da aplicação e separar regras de negócio da interface visual.

7. O que é e como é usado o Material Design e o Cupertino em Flutter?

Material Design é a identidade visual do Google (usada no Android), enquanto **Cupertino** é a identidade da Apple (iOS). O Flutter oferece bibliotecas de widgets para ambas (`material.dart` e `cupertino.dart`), permitindo que o desenvolvedor crie interfaces com a aparência e comportamento nativo de cada plataforma, ou até mesmo misture ambas.

8. O que é .NET MAUI? Compare com o Flutter e Android.

.NET MAUI é o framework multiplataforma da Microsoft (usando C#). **Comparação:** O Android nativo é exclusivo para Android. O Flutter (Dart) desenha sua própria interface independente do sistema. Já o MAUI (C#) traduz o código para usar os componentes de interface nativos reais do sistema operacional subjacente.

9. Qual a diferença entre Hot Reload e Hot Restart?

Hot Reload: Atualiza apenas o código modificado na interface em milissegundos, *preservando o estado atual* do app. **Hot Restart:** Reinicia a aplicação por completo, reconstruindo a árvore de widgets do zero e *perdendo o estado atual*. É usado quando há mudanças estruturais profundas.

10. Explique o que faz o código abaixo:

```
void main() => runApp( MaterialApp( title: 'Whatsapp Clone', home: Scaffold(
  body: FlutterLogo( size: double.infinity, ) ) ) );
```

O código inicializa o app (`main()` e `runApp()`) configurando um ambiente Material (`MaterialApp`) chamado 'Whatsapp Clone'. A tela inicial (`home`) é um `Scaffold` que exibe apenas o logotipo do Flutter (`FlutterLogo`) no centro da tela, forçando-o a ocupar todo o espaço disponível através da propriedade `size: double.infinity`.

11. O que é e qual a diferença entre Provider e Riverpod?

Ambos gerenciam o estado no Flutter. O **Provider** depende da Árvore de Widgets para funcionar; se tentar acessar um dado onde ele não foi declarado, o app quebra (erro de execução). O **Riverpod** (do mesmo criador) resolve isso sendo independente da árvore de widgets, garantindo segurança em tempo de compilação (compile-safe) e evitando quebras no app.

12. Explique o Widgets Tree da imagem 1.

A **Widgets Tree** (Árvore de Widgets) representa a hierarquia do layout. O `MyApp` inicia o `MaterialApp`. A página `MyHomePage` usa um `Scaffold` que se divide em 3 braços:

1. `AppBar` no topo com um `Text`.
2. `Center` no corpo com uma `Column` que empilha verticalmente dois `Text` (textos na tela).
3. `FloatingActionButton` contendo um `Icon`.

13. Qual o objetivo de um emulador?

Um emulador simula o sistema operacional de um dispositivo móvel (Android ou iOS) no computador do desenvolvedor. Seu objetivo é permitir testar e debugar o aplicativo visualmente sem a necessidade de conectar um celular físico via cabo USB.

14. De acordo com a estrutura de desenvolvimento, o que são main.dart e runApp()?

O `main.dart` é o arquivo principal e ponto de entrada da aplicação. O Flutter sempre executa esse arquivo primeiro. A função `runApp()` é o comando que pega o widget principal do seu código e o "injeta" na tela do dispositivo, iniciando de fato a interface visual do app.

15. Os widgets podem ser classificados em Stateless e Stateful, o que são e qual a diferença?

Stateless (Sem Estado): Widgets imutáveis. Depois de desenhados, sua aparência não muda por conta própria (ex: um ícone fixo).

Stateful (Com Estado): Widgets dinâmicos. Eles possuem memória interna e podem se redesenhar na tela (usando `setState()`) em resposta a cliques ou mudanças de dados (ex: uma caixa de marcação - checkbox).

16. O que são Scaffold e quais são (suas propriedades)?

Scaffold é um widget estrutural que fornece um "esqueleto" padrão de tela (gavetas, barras e corpo). Suas principais propriedades são: **appBar** (barra superior), **body** (conteúdo central da tela), **floatingActionButton** (botão flutuante), e **bottomNavigationBar** (menu inferior).

17. Explique o que é cada componente dessa interface mobile da imagem.

Com base no diagrama da interface:

- **MaterialApp**: Engloba todo o app, configurando regras globais.
- **Scaffold**: Estrutura física daquela tela específica.
- **AppBar**: A barra superior de título contendo "Simple Flutter App".
- **Body**: Toda a área branca e livre no centro da tela.
- **Button / Text**: Componente clicável no centro do "body", exibindo internamente o texto "Hello, Flutter!".

20. Cite os requisitos para o desenvolvimento mobile usando flutter para o Android e para o iOS.

Base comum: Instalar o Flutter SDK e uma IDE (como VS Code ou Android Studio).

Para Android: É necessário o Android Studio (para instalar o Android SDK e Emuladores). Pode ser feito no Windows, Linux ou Mac.

Para iOS: É obrigatório ter um computador com **macOS** (um Mac). Exige a instalação do **Xcode** (IDE da Apple) e do utilitário *CocoaPods* para que o app seja compilado para iPhones ou iPads.